

Module 5: Spring Boot Basics

BHEL INTERNSHIP PROGRAM

Location: Bharat Heavy Electricals Limited

1. Introduction to Spring Boot

Spring Boot is an extension of the Spring framework designed to simplify the initial setup and development of new Spring applications. It adopts an "opinionated" view of the Spring platform, allowing developers to build stand-alone, production-grade applications with minimal boilerplate configuration. For industrial environments like BHEL, it is used to rapidly develop microservices that handle complex data processing tasks.

2. Features of Spring Boot

- **Auto-Configuration:** Automatically configures the application based on the dependencies present in the classpath.
- **Starter Dependencies:** Simplifies build configuration by providing "starter" descriptors (e.g., `spring-boot-starter-web`).
- **Embedded Servers:** Includes embedded servers like Tomcat or Jetty, eliminating the need to deploy WAR files to external servers.
- **Actuator:** Provides production-ready features such as health checks and metrics for monitoring application performance.
- **Standalone:** Applications can be run as a standard Java application using a simple `main` method.

3. Project Structure

A standard Spring Boot project follows a layered architecture to maintain a clear separation of concerns, which is essential for managing large-scale industrial software.

Folder / Package	Responsibility
<code>src/main/java</code>	Contains all Java source code packages.

Folder / Package	Responsibility
<code>.controller</code>	Maps HTTP requests to service methods.
<code>.service</code>	Contains business logic and orchestrates data flow.
<code>.repository</code>	Handles database operations via Spring Data JPA.
<code>.model</code>	Defines data entities (POJOs).
<code>src/main/resources</code>	Contains configuration (<code>application.properties</code>) and static assets.

4. Controllers

Controllers are responsible for processing incoming web requests. In a web application using Thymeleaf, the `@Controller` annotation is used to return views (HTML templates).

```
@Controller
public class AssetController {
    @Autowired
    private AssetService service;

    @GetMapping("/assets")

    public String viewAssets(Model model) {

        model.addAttribute("assets", service.getAllAssets());

        return "asset-list"; // Refers to asset-list.htm
    }
}
```

5. Services

The Service layer, annotated with `@Service`, holds the core business logic of the application. It acts as an intermediary between the controller and the repository.

```
```java
```

```
@Service
```

```
public class AssetService {
```

```
 @Autowired
```

```
 private AssetRepository repository;
```

```
 public List<Asset> getAllAssets() {
```

```
 return repository.findAll();
```

```
 }
```

```
}
```

## 6. Repositories

Repositories are interfaces that extend `JpaRepository` to provide standard CRUD (Create, Read, Update, Delete) operations without requiring manual implementation code. `@Repository`

```
public interface AssetRepository extends JpaRepository<Asset, Long> {
```

```
}
```

## 7. Models

Models are Plain Old Java Objects (POJOs) that represent the data structure, often mapped to database tables using the `@Entity` annotation.

```
@Entity
```

```
public class Asset {
```

```
 @Id
```

```
 @GeneratedValue(strategy = GenerationType.IDENTITY)
```

```
 private Long id;
```

```
 private String name;
```

```
private String unitLocation;
// Getters and Setters
}
```

## ## 8. Thymeleaf Templates

Thymeleaf is a modern server-side Java template engine used to create dynamic web pages. Templates are typically stored in `src/main/resources/templates`.

```
``html

<!DOCTYPE html>

<html xmlns:th="http://www.thymeleaf.org">

<body>

 <h1>BHEL Asset List</h1>

 <table>

 <tr th:each="asset : ${assets}">

 <td th:text="${asset.name}">Name</td>

 <td th:text="${asset.unitLocation}">Location</td>

 </tr>

 </table>

</body>

</html>
```

## 9. Database Connectivity using MySQL

To connect a Spring Boot application to a MySQL database, configure the connection details in the `src/main/resources/application.properties` file.s

```
spring.datasource.url=jdbc:mysql://localhost:3306/bhel_db
spring.datasource.username=root
spring.datasource.password=password
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
```

## ## 10. Building a Simple Web Application

To build a basic asset tracking system:

1. **Define the Model:** Create an `Asset` entity with fields like ID, Name, and Location.
2. **Create the Repository:** Extend `JpaRepository` for data access.
3. **Implement the Service:** Add logic to fetch and save assets.
4. **Develop the Controller:** Use `@Controller` to handle GET requests and pass data to the view.
5. **Design the View:** Use Thymeleaf to display the asset data in an HTML table.
6. **Configure MySQL:** Set up the database properties to ensure persistent storage.

Congratulations! You have completed Module 5.

<span type="placeholder" placeholder-type="person"></span>

Instructor, BHEL-HRDC

<span type="placeholder" placeholder-type="date"></span>